INSPIRE AI

STAND-UP

Standup Meeting



© 2026 Inspire | Confidential

One Check That Changed Your Mind

O • Objective	What happened? What did you observe?
R • Reflective	How did you feel about that experience?
I • Interpretive	What did it help you understand?
D • Decisional	What will you try today?

Share one real experience from yesterday. Take equal turns. Choose how much personal feeling to share. Work without AI today.

© 2026 Inspire | Confidential

REVIEW

Share Example Tests and Peer Comments



© 2026 Inspire | Confidential

Your Test, Your Example, Your Result

Open your graded code and tests. Explain which examples they check and what happened when you ran them.

Example	Which confirmed example does this test check?
Assertion	Which result does it check? Why is that value expected?
Run	What happened when you ran the submitted tests?

Every member presents. Divide all 20 minutes equally, including feedback. Support one useful comment with a rule or the shown code.

© 2026 Inspire | Confidential

YOUR TURN

When a Check Finds a Problem, What Happens Next?

Compare A and B • Explain the next action

Suppose the check reports a problem in Change 1. What has each process already changed?



© 2026 Inspire | Confidential

Do These Checks Cover the New Request?

Existing rule: members get a 10% discount.

Existing check	Expected discount	Result
Non-member • Small	0%	Pass
Member • Small	10%	Pass

> **New rule:** a large cup adds 5%. Discounts add together.

Do these checks tell us that a member gets 15% for a large cup?

Illustrative coffee example. Results are a teaching illustration, not a recorded run.

© 2026 Inspire | Confidential

Choose One Goal from the Test List

The small-cup checks already pass. Add the two large-cup goals.

Behavior	Expected discount	Status
Non-member • Small	0%	Existing check passes
Member • Small	10%	Existing check passes
Member • Large	15%	Still to check ← Select this
Non-member • Large	5%	Still to check

Illustrative status. We will keep this list and update it after checking each goal.

© 2026 Inspire | Confidential

Prepare a Runnable Check Before Coding

One useful arrangement: make the next goal runnable before changing the implementation.

Choose a goal

Member • Large
15% discount

→

Write a check

Use the agreed input and expected result.

→

Change and check

The check is ready when the change is ready.

Run it now as well: does it report the gap in the current behavior?

© 2026 Inspire | Confidential

Write the Goal as a Call and an Expected Result

Member 10% + Large 5% = **15%**

Illustrative pseudocode • Existing interface

```
assert discount(member=true, large=true) == 15
```

Input	A member buying a large cup.
Current code	Returns 10 for a member, 0 otherwise. Cup size is unused.

Before any implementation change: what result will the current implementation return? Why?

© 2026 Inspire | Confidential

What Does This Result Tell Us?

Same check • Before changing the implementation

```
assert discount(member=true, large=true) == 15
```

Expected

15%

From the agreed rule

Actual

10%

From the current implementation

Illustrative result. Why was 15% expected, and why did this implementation return 10%?

© 2026 Inspire | Confidential

Read the Failure Before Changing the Code

Expected: 15%	The rule adds the member discount and the large-cup discount.
Actual: 10%	The current implementation applies only the member discount.
Decision	The check reports the large-cup gap. We have a reason to change the implementation.

This is a behavior Red. An environment failure needs diagnosis before a business-code change.

© 2026 Inspire | Confidential

Choose the Change Scope and the Checks

This step: make the member large-cup check pass while preserving existing behavior.

Possible implementation work

Large-cup discount
Amount rounding
Pricing structure

Checks on our list

Selected member large-cup check
Existing small-cup checks
Remaining non-member large-cup check

What belongs in this change? Which checks must pass before this step can finish?

Keep the agreed expectations. Choose the scope and validation before seeing the implementation.

© 2026 Inspire | Confidential

Choose the Scope, Then Implement

This step: large-cup discount

If it fails, examine this change and its check.

Also change rounding and structure?

More changes would need their effects explained.

Illustrative pseudocode • Add the large-cup behavior only

```
discount = member ? 10 : 0
if (large) discount += 5
return discount
```

Run the new check and the applicable existing checks now, before making another change.

© 2026 Inspire | Confidential

What Does the New Run Show?

Same check • After the implementation change

```
assert discount(member=true, large=true) == 15
```

Expected

15%

Actual

15%

Existing small-cup checks: 0% and 10% still pass.

Illustrative results. What do these checks support? What is still on our test list?

© 2026 Inspire | Confidential

Inspect This Code While Tests Are Green

Same illustrative implementation

```
discount = member ? 10 : 0
if (large) discount += 5
return discount
```

Inspect the structure

Are the names clear?

Is there repeated calculation?

Is unrelated work mixed in?

This round: no useful structural change is needed. Keep the code as it is.

Return to the list: Non-member • Large → 5% is still to check.

© 2026 Inspire | Confidential

What Can We Decide from These Records?

Only checked after

After the change:

15% • **Pass**

Checked before and after

Before: **10%** • **Fail**

After: **15%** • **Pass**

What was known before the change? What helps us decide whether to continue after it?

Both records use the same check: member, large cup, expected discount 15%. Illustrative results.

© 2026 Inspire | Confidential

Start Implementation with a Check Already in Use

Before the change

The agreed rule gives the expected 15%.
Before coding, this check has run: actual 10% exposes the current gap.

→ **Use this available check to guide the implementation.**

After the change

Keep the same check and run it immediately: expected 15%, actual 15%. Existing checks also pass.

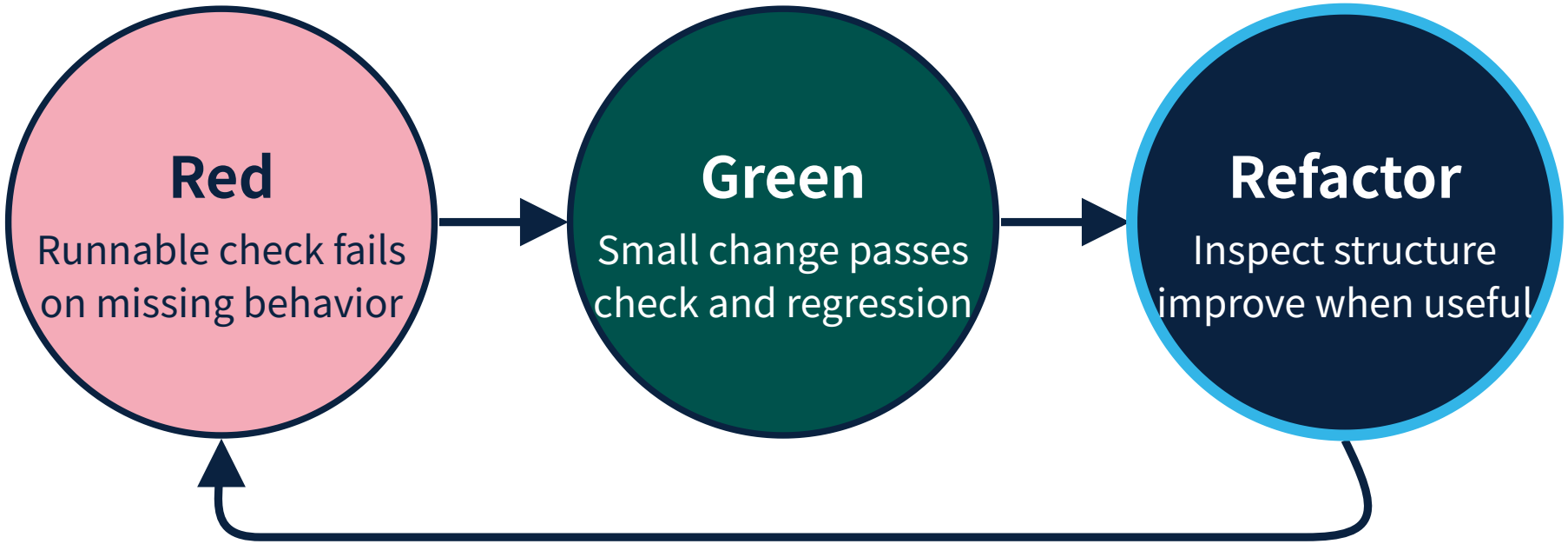
→ **Inspect structure, then choose the next pending item.**

This is one useful arrangement, not the only way to obtain early feedback.

© 2026 Inspire | Confidential

TDD Repeats a Red–Green–Refactor Cycle

TDD: Test-Driven Development



Choose one behavior. Use each result to decide what to do next. Then return to the test list.

© 2026 Inspire | Confidential

Red → Green: Explain the Behavior Gap

> Coffee: Expected 15%, Actual 10%. The assertion reports the large-cup gap.

Ready to implement?	The expected behavior has an agreed rule. The check runs, and we can explain its failure.
Continue when	The failed assertion corresponds to the behavior we intend to add.

Environment failure: diagnose first. Already passing: review the check and current behavior; do not break the code to force Red.

© 2026 Inspire | Confidential

Green → Refactor: Check This Step and Regression

> Coffee: the member large-cup check passes, and the earlier small-cup checks still pass.

Keep the scope	Make the smallest complete implementation change for the selected behavior.
Continue when	The selected check and applicable existing checks pass. Then inspect structure.

If a check still fails, read the cause. Keep correct expectations and the remaining test list.

© 2026 Inspire | Confidential

Refactor → Next Item: Review Structure and Keep Green

> Coffee: we inspected these few lines and found no useful structural change. Keep them.

If improvement helps	Change structure in small steps, preserve behavior, and run checks again.
Return to the list when	The structure review is complete and checks stay green.

A failed structural change needs correction first. Adding a discount rule is new behavior, not a refactor.

© 2026 Inspire | Confidential

Return to the Same Test List

Behavior	Expected discount	Status
Non-member • Small	0%	Existing check passes
Member • Small	10%	Existing check passes
Member • Large	15%	This step passes
Non-member • Large	5%	Still to check

Requirements and criteria → Examples → Test list → Next useful check

Illustrative status. “Still to check” does not mean “not yet implemented”.

© 2026 Inspire | Confidential

Let the Next Check Choose the Path

Illustrative next check

```
assert discount(member=false, large=true) == 5
```

Fails on missing behavior

Explain the gap.
Implement the next small step.

Already passes

Review the input and assertion.
Keep it as regression and return to the list.

Keep expectations aligned: retain unchanged rules. If an approved rule replaces behavior, update only its affected assertions before implementation.

The illustrated code suggests this may pass. It remains unrecorded until it runs.

© 2026 Inspire | Confidential

What Does Each Action Help Us Do?

Test first	Prepare and run the member large-cup check before implementation.
Small steps	Add the large-cup discount without changing rounding or pricing structure.
Frequent checks	Run and read the result before starting another change.

What does each action contribute to useful feedback?

Use the coffee process to explain each action.

© 2026 Inspire | Confidential

Each Action Supports a Different Part of Feedback

Test first	The goal check is ready when implementation begins. We have seen its response to the current gap.
Small steps	Keep the unverified change limited, so its effect is easier to examine.
Frequent checks	Get and use the result before further work accumulates.

Writing early does not make a check run early. Running often does not make a change smaller.

© 2026 Inspire | Confidential

How Would You Arrange the Next Change?

Both developers first wrote and ran a check for the intended behavior.

Process A

Changes one behavior at a time.
Runs tests only after several changes.

Process B

Changes the discount, rounding and pricing structure together.
Runs and reads tests immediately. A check fails.

What would you change in each process? What problem would that solve?

Think independently, then explain your reasons.

© 2026 Inspire | Confidential

Use Feedback Before Work Accumulates

Process A	Run and read checks before starting the next item. Small edits can still accumulate into one unverified batch.
Process B	Limit the scope of one change. An immediate check does not make that change smaller.

Prepare the check → Change a small step → Run and read → Decide

Next: read the order requirement and write your own examples. A check needs an expected result with a rule source.

© 2026 Inspire | Confidential

BREAK

Take a Break



© 2026 Inspire | Confidential

INSPIRE AI

PRACTICE

Clarify the Criteria and Write Examples



© 2026 Inspire | Confidential

The Customer's Goal

As a	customer placing a larger order
I want	**a higher promotion at each qualifying amount**
So that	my quote uses the correct promotion band

Source story: US-04

© 2026 Inspire | Confidential

The Complete Request

Use the price after the member discount to select one promotion: 20 yuan at 100 yuan, 40 yuan at 300 yuan, and 80 yuan at 500 yuan. Apply only the highest qualifying band once. Keep the other rules unchanged.

© 2026 Inspire | Confidential

The Promotion Bands

Select one promotion using the member price. Amounts in yuan.

Member price	Promotion
Below 100	0
100 to below 300	20
300 to below 500	40
500 or more	80

US-04-AC01: Apply the highest qualifying band once. Do not add bands together.

© 2026 Inspire | Confidential

The Amount That Selects the Band

Follow the calculation order to select the promotion band.

Use to select the band	Do not use to select the band
Member price after discount and rounding	Original price or goods payable after promotion

US-04-AC02: Calculate the member price first. Then select the promotion and calculate goods payable.

Write your own examples in your existing notes: specific inputs, an action, expected results and their rule source. Work independently for 11 minutes before viewing the reference.

© 2026 Inspire | Confidential

INSPIRE AI

REVIEW

Compare Examples and Confirm Expected Results



© 2026 Inspire | Confidential

Keep the Existing Promotion

Given	Original price **250.00**	Member **No**	
When	Calculate the quote.		
Then Expected result	Original price **250.00**	Member price **250.00**	Promotion **20.00**
	Goods payable **230.00**	Shipping fee **0.00**	Total payable **230.00**

Observation: Not recorded

Source example: REVIEW-PROMOTION-E02

Business example 1 of 4: the existing promotion still applies.

© 2026 Inspire | Confidential

Apply the Promotion at 300 Yuan

Given	Original price **300.00**	Member **No**	
When	Calculate the quote.		
Then Expected result	Original price **300.00**	Member price **300.00**	Promotion **40.00**
	Goods payable **260.00**	Shipping fee **0.00**	Total payable **260.00**

Observation: Not recorded

Source example: US-04-E02

Business example 2 of 4: reaching 300 yuan qualifies for the new promotion band.

© 2026 Inspire | Confidential

Apply Only the Highest Promotion Band

Given	Original price **500.00**	Member **No**	
When	Calculate the quote.		
Then Expected result	Original price **500.00** Goods payable **420.00**	Member price **500.00** Shipping fee **0.00**	Promotion **80.00** Total payable **420.00**

Observation: Not recorded

Source example: US-04-E03

Business example 3 of 4: use only the highest qualifying band. Do not add promotions together.

© 2026 Inspire | Confidential

Use the Price After the Member Discount

Given	Original price **300.00**	Member **Yes**	
When	Calculate the quote.		
Then Expected result	Original price **300.00**	Member price **240.00**	Promotion **20.00**
	Goods payable **220.00**	Shipping fee **0.00**	Total payable **220.00**

Observation: Not recorded

Source example: US-04-E04

Business example 4 of 4: confirm the amount that selects the band. Next, analyze which tests are needed.

© 2026 Inspire | Confidential

Find the Input, Action and Expected Result

Given	Original price **300.00**	Member **No**	
When	Calculate the quote.		
Then Expected result	Original price **300.00**	Member price **300.00**	Promotion **40.00**
	Goods payable **260.00**	Shipping fee **0.00**	Total payable **260.00**

Observation: Not recorded

Source example: US-04-E02

Use this confirmed example: which values are inputs, what operation runs, and which returned values can a test observe?

© 2026 Inspire | Confidential

The Same Example Can Become a Test

Given	Supply the confirmed input.
When	Call the relevant operation.
Then	Assert the expected observable result.

Keep the same notes. Reuse the existing nonmember 250 check. The other three business examples need checks if coverage is absent.

© 2026 Inspire | Confidential

A Useful Test Has a Clear Purpose

> What mistake could this check reveal?

- Make each expected result traceable to a rule.
- Keep the purpose clear when combining assertions.
- Group checks when that makes the test easier to understand.

Example count and test-method count can differ.

© 2026 Inspire | Confidential

An Equivalence Class Shares a Rule

Group inputs that the same rule handles. A representative helps reduce repeated work.

Ask	Which possible implementation mistake matters here?
Compare	Would a different rule produce a different expected result?

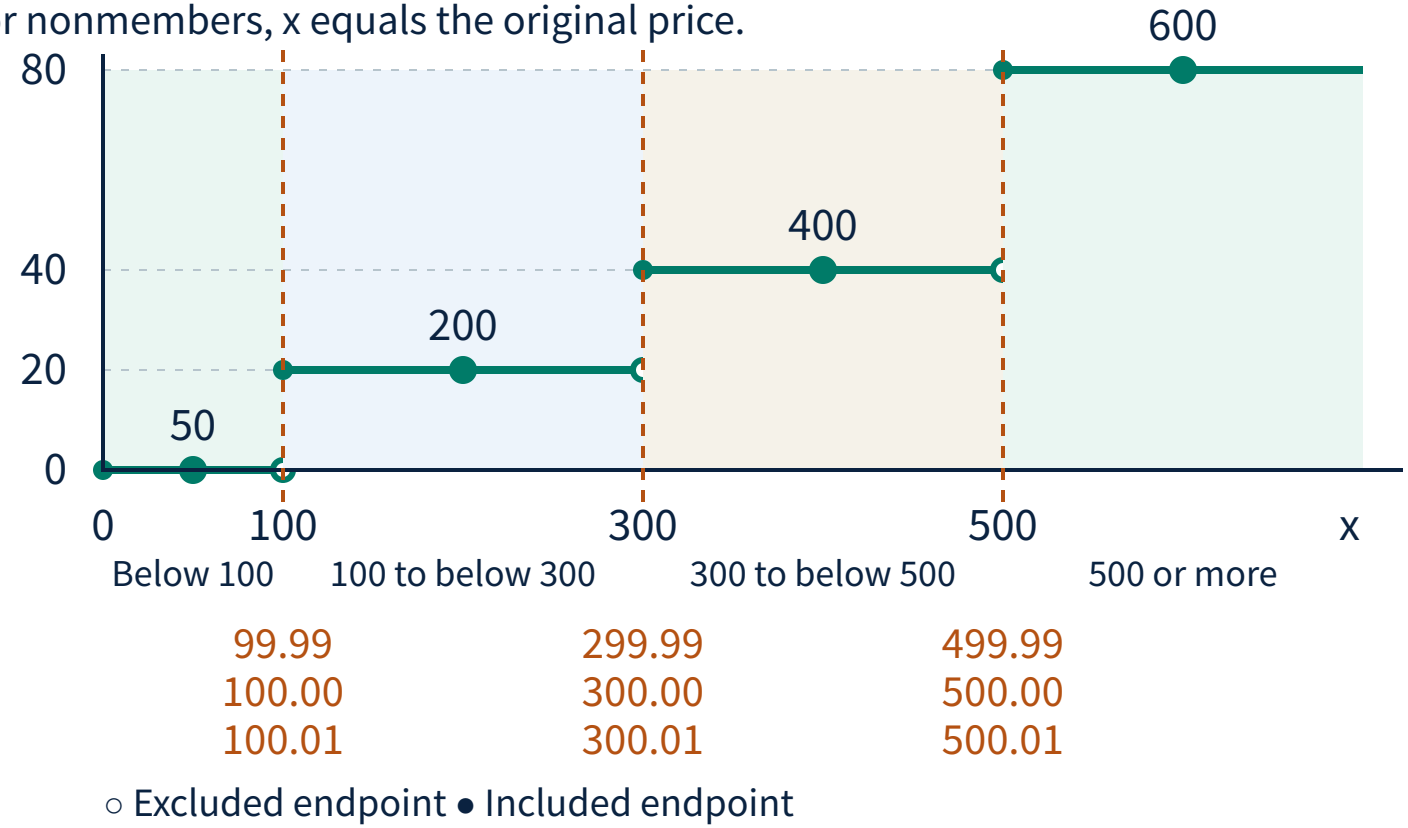
Which bands already have checks? Which new bands do the 300 and 500 business examples cover? A representative cannot prove the whole group correct.

© 2026 Inspire | Confidential

Equivalence Classes and Boundary Values

US-04 • x = price after member discount and rounding • y = promotion (yuan)

For nonmembers, x equals the original price.



Equivalence partitioning

One representative per band

Same band, same promotion

Boundary value analysis

Below, at and above each cut

Check where the band changes

Nearby amounts differ by 0.01 yuan.

x is the price after discount.

US-04-AC01: Apply the highest qualifying band once. Do not add bands together.

© 2026 Inspire | Confidential

Which Boundary Checks Are Missing?

The business examples already include exactly 300 and 500 yuan.

Derive	Choose the closest amount below and above each new threshold.
Predict	State the expected promotion and the mistake each check could detect.
Reuse	Keep the checks at each threshold. Do not add them twice.

Use one cent as the adjacent unit. Explain your choices before viewing the reference checks.

© 2026 Inspire | Confidential

Add Two Checks Around 300

Given the original price and membership status, calculate the quote. Amounts in yuan.

Example	Original	Member?	Member price	Promotion	Goods payable	Shipping	Total payable
US-04-E01	299.99	No	299.99	20.00	279.99	0.00	279.99
US-04-E05	300.01	No	300.01	40.00	260.01	0.00	260.01

Expected results. Expected original price equals the input. Actual results: Not recorded.

Keep the existing business example at 300. These two additions check below and above it.

© 2026 Inspire | Confidential

Add Two Checks Around 500

Given the original price and membership status, calculate the quote. Amounts in yuan.

Example	Original	Member?	Member price	Promotion	Goods payable	Shipping	Total payable
US-04-E06	499.99	No	499.99	40.00	459.99	0.00	459.99
US-04-E07	500.01	No	500.01	80.00	420.01	0.00	420.01

Expected results. Expected original price equals the input. Actual results: Not recorded.

Keep the business example at 500. Three business checks plus four boundary checks now need coverage.

© 2026 Inspire | Confidential

Can This Example Detect the Wrong Amount?

Given	Original price **300.00**	Member **Yes**	
When	Calculate the quote.		
Then Expected result	Original price **300.00**	Member price **240.00**	Promotion **20.00**
	Goods payable **220.00**	Shipping fee **0.00**	Total payable **220.00**

Observation: Not recorded

Source example: US-04-E04

Trace the band using original price, then member price. Would using the wrong amount produce a different promotion? Explain the possible error before choosing another check.

© 2026 Inspire | Confidential

Which Member Combinations Are Missing?

Combine membership with the promotion band after the discount.

Member-price band	Nonmember coverage	Member coverage
Earlier bands	Existing tests	Existing tests and the business example
New middle band	Business and boundary checks	Which input reaches this band?
New highest band	Business and boundary checks	Which input reaches this band?

Derive the member original prices before viewing the next page. Member price is calculated from the inputs; it is not an independent input.

© 2026 Inspire | Confidential

Add Member Checks for Both New Bands

Given the original price and membership status, calculate the quote. Amounts in yuan.

Example	Original	Member?	Member price	Promotion	Goods payable	Shipping	Total payable
US-04-E08	375.00	Yes	300.00	40.00	260.00	0.00	260.00
US-04-E09	625.00	Yes	500.00	80.00	420.00	0.00	420.00

Expected results. Expected original price equals the input. Actual results: Not recorded.

These two additions check the member-discount path into each new band. Reuse them if they are already in your own list.

© 2026 Inspire | Confidential

Mark Reused Checks and Nine Additions

Source	Action	Reference checks
Earlier business example	Reuse 1	REVIEW-PROMOTION-E02 / US-02-E10
Other business examples	Retain 3 additions	US-04-E02, E03, E04
Adjacent boundaries	Add 4	US-04-E01, E05, E06, E07
Member combinations	Add 2	US-04-E08, E09

Spend three minutes updating your notes. The complete reference baseline needs 9 additional input/result checks. Reuse personal coverage, remove only genuine duplicates and keep every required input. Method and TDD-cycle counts may differ.

© 2026 Inspire | Confidential

Tasking Starts with the Existing Test List

Choose	Select a useful behavior that is not yet satisfied.
Develop	Turn it into a runnable check and use its feedback.
Revisit	Keep new discoveries and remaining checks on the same list.

Select the next small behavior. Update the order after each result.

© 2026 Inspire | Confidential

An Increment Describes Observable Behavior

Creating a class, writing a method and running tests support a behavior increment.

Split when needed	A large item can become smaller independently checkable behaviors.
Keep the purpose	Technical work should identify the behavior it supports.
Refactor while Green	A structural improvement preserves the checked behavior.

Those technical actions are not separate business increments or separate TDD cycles. A clear small test case needs no second task document.

© 2026 Inspire | Confidential

Your Next Useful Check

> Mark one next item in your current list. Explain the gap it addresses.

Keep the remaining boundary, membership and preserved-behavior checks visible.

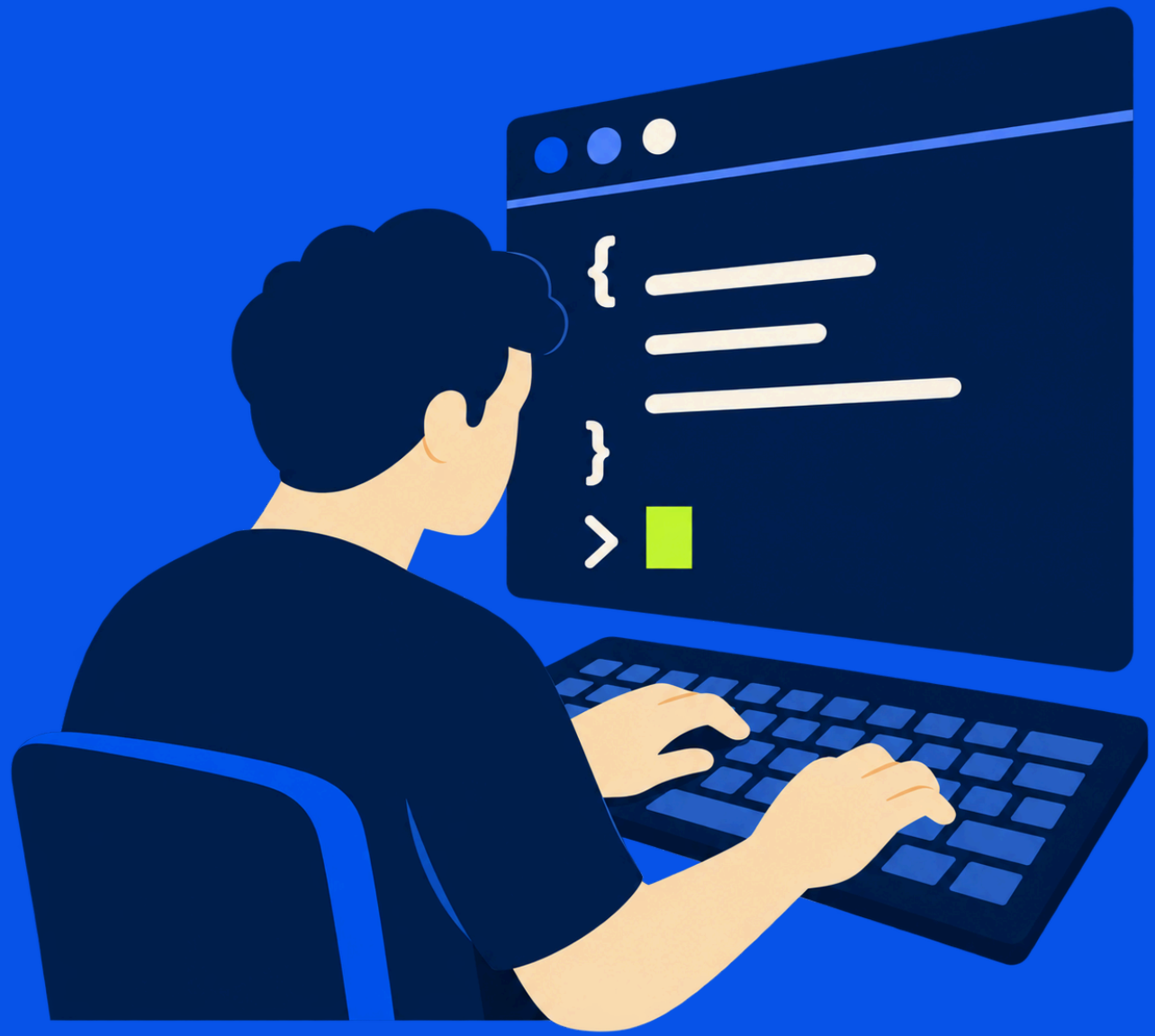
Use the final minute. The demonstration is one example of a cycle. Your later choices depend on your own results.

© 2026 Inspire | Confidential

INSPIRE AI

DEMO

Demonstrate the First Test-Driven Change



© 2026 Inspire | Confidential

The Selected Behavior and Its Expected Quote

Given	Original price **300.00**	Member **No**	
When	Calculate the quote.		
Then Expected result	Original price **300.00**	Member price **300.00**	Promotion **40.00**
	Goods payable **260.00**	Shipping fee **0.00**	Total payable **260.00**

Observation: Not recorded

Source example: US-04-E02

Use this agreed expectation during the live demonstration in the IDE and terminal.

© 2026 Inspire | Confidential

BREAK

Take a Break



© 2026 Inspire | Confidential

INSPIRE AI

COACHING

Develop from Your Test List with Guidance



© 2026 Inspire | Confidential

Start from Your Own Project

Select useful examples, run tests and use feedback to drive small implementation and structural decisions with coaching.

1. Open your personal Day 3 Homework clone with the supplied baseline tests. Run the suite and choose a useful missing check.
2. Write or reuse the test, run it before production, then explain the result.
3. Implement missing agreed behavior, check regression and choose the next useful item.

Produce

selected personal tests, expected results and actual outcomes mapped to examples

current personal code, actual development evidence and remaining test-list items

20 minutes • whole activity

Source activity: d03-s08

Work without AI. Use your current code, examples and test list; choose from your actual progress.

© 2026 Inspire | Confidential

INSPIRE AI

PRACTICE

Continue Test-Driven Work with Less Guidance



© 2026 Inspire | Confidential

Continue from Your Current Progress

Independently choose and complete further useful checks and changes from the remaining test list.

1. Continue the same project and choose the next useful check.
2. Use test feedback to implement missing behavior and check all 9 incremental US-04 inputs and reuse earlier regression tests.
3. Run the full applicable suite and save the tested version, real results and gaps.

Produce

independent test-list progress, actual implementation and regression results, help and remaining gaps

35 minutes • whole activity

Source activity: d03-s09

Continue directly from the supported practice. Keep all 9 reference inputs and earlier regression in scope; save real results and one decision for the showcase.

© 2026 Inspire | Confidential

BREAK

Lunch Break



© 2026 Inspire | Confidential

MINI-GAME

Optional Mini-game



© 2026 Inspire | Confidential

INSPIRE AI

SHOWCASE

Show Test-Driven Decisions and Results



© 2026 Inspire | Confidential

Show a Decision That Changed Your Work

Two real experiences: one new failure drives a small implementation; the other changes the next choice or passes immediately.

Choice		Feedback		Next action
The example and its rule	→	The actual test result	→	What you changed or checked

About five minutes each, including feedback; compare for two minutes. Show the saved version and real result. Use another genuine decision when the suggested experience did not happen.

© 2026 Inspire | Confidential

BREAK

Take a Break



© 2026 Inspire | Confidential

Make the Next Change Easier to Understand

Refactoring improves internal structure while preserving observable behavior.

Start Green

Check current behavior

→

Improve structure

One clear benefit

→

Verify and save

Tests, diff and commit

A new fee rule changes behavior. A clearer name or calculation boundary can preserve it. Read the small diff as well as the test result.

© 2026 Inspire | Confidential

When to Refactor

Code review

Review with the author. Improve unclear code before sharing it.

Adding a feature

Clarify the code you need to change. Keep each structural step Green.

Fixing a defect

Understand the failure. Separate behavior changes from structural changes.

Rule of three

On a third similar use, consider sharing the repeated logic.

Confirm that the code represents the same rule before combining it.

© 2026 Inspire | Confidential

Which Role Does This Fee Name Express?

QuoteCalculator.java java • checkpoint/quote-with-day2-tests • lines 13–17

```
13          BigDecimal shippingBasis = goods;
14          BigDecimal shippingThreshold = Money.amount("100.00");
15          boolean freeShipping = shippingBasis.compareTo(shippingThreshold) >= 0;
16          BigDecimal deliveryFee = Money.amount("10.00");
17          BigDecimal shipping = freeShipping ? Money.amount("0.00") : deliveryFee;
```

src/main/java/training/quote/QuoteCalculator.java

Is deliveryFee the charged rate or the final amount? Follow its use to decide.

Earlier Day 2 baseline. A vague name makes the reader trace the condition before understanding the value's role.

© 2026 Inspire | Confidential

Which Edits Would Need to Stay in Sync?

QuoteCalculator.java java • checkpoint/quote-with-day2-tests • lines 18–23

```
18          if (member) {
19              BigDecimal memberTotal = goods.add(shipping);
20              return new Quote(original, memberPrice, promotion, goods, shipping,
    memberTotal);
21          }
22          BigDecimal standardTotal = goods.add(shipping);
23          return new Quote(original, memberPrice, promotion, goods, shipping,
    standardTotal);
```

src/main/java/training/quote/QuoteCalculator.java

If the Quote result gains a field, which two return paths must change together?

Earlier Day 2 baseline. Both paths assemble the same result; the morning demonstration removes this duplication. Similar-looking code can also represent different rules.

© 2026 Inspire | Confidential

Read the Price and Promotion in calculate

QuoteCalculator.java java • checkpoint/quote-with-day2-tests • lines 8–12

```
8           }
9           BigDecimal memberPrice = Money.memberPrice(original, member);
10          BigDecimal promotion = memberPrice.compareTo(Money.amount("100.00")) >= 0
11                  ? Money.amount("20.00") : Money.amount("0.00");
12          BigDecimal goods = memberPrice.subtract(promotion);
```

src/main/java/training/quote/QuoteCalculator.java

This part finds the goods amount. What responsibility comes next?

Earlier Day 2 baseline. Keep this purpose in mind while reading the next part of the same method.

© 2026 Inspire | Confidential

Shipping Details in the Same Method

QuoteCalculator.java java • checkpoint/quote-with-day2-tests • lines 13–17

```
13          BigDecimal shippingBasis = goods;
14          BigDecimal shippingThreshold = Money.amount("100.00");
15          boolean freeShipping = shippingBasis.compareTo(shippingThreshold) >= 0;
16          BigDecimal deliveryFee = Money.amount("10.00");
17          BigDecimal shipping = freeShipping ? Money.amount("0.00") : deliveryFee;
```

src/main/java/training/quote/QuoteCalculator.java

Shipping details interrupt the quote's price and promotion flow.

A code smell suggests a reading or editing difficulty to investigate. It does not prove a defect. Source: Fowler, Code Smell.

© 2026 Inspire | Confidential

Choose a Small Change with a Clear Benefit

Benefit	Which current reading or change difficulty would improve?
Scope	What is the smallest coherent change that addresses it?
Evidence	Which applicable tests and diff will help check behavior?

Keep acceptable structure when no concrete difficulty exists. Diagnose business failures and missing checks before structural changes.

© 2026 Inspire | Confidential

Connect Each Operation to Its Purpose

Rename	Give a value or action a name that explains its role.
Extract Method	Name one complete calculation: shippingFor(goods) reads goods and returns shipping.
Compare calls	shippingFor(goods) names the work. step2(goods) makes the reader open another method to discover it.

Illustrative names for the same calculation. A shorter method can add a jump without explaining the work.

© 2026 Inspire | Confidential

Which Change Would You Make First?

Candidate A	Clarify the name of the amount charged when shipping applies.
Candidate B	Give the complete shipping calculation its own method.

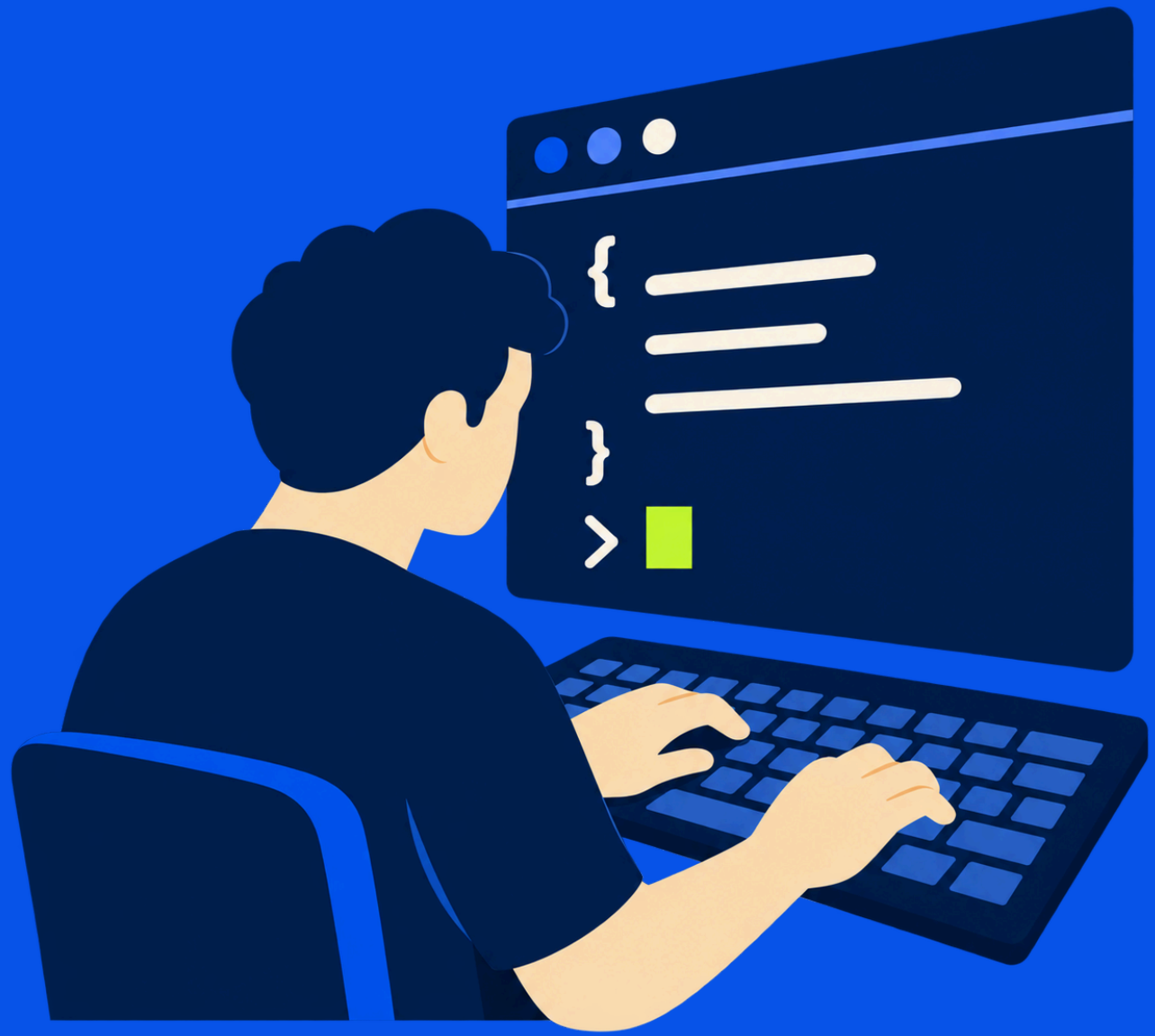
> Choose a first step and explain why. What information is still missing?

© 2026 Inspire | Confidential

INSPIRE AI

DEMO

Demonstrate Rename and Method Extraction



© 2026 Inspire | Confidential

Shipping Calculation: Input and Result

goods

Read by the current calculation

→

shippingFor

Calculate the shipping amount

→

shipping

Return to calculate

Derive parameters from the values the code actually reads.

The current calculation reads only goods. It returns shipping to calculate. Recheck dependencies when a rule changes.

© 2026 Inspire | Confidential

BREAK

Take a Break



© 2026 Inspire | Confidential

PRACTICE

Practice Rename and Method Extraction



© 2026 Inspire | Confidential

Find Two Useful Changes in Your Own Code

Practice a useful local rename and method extraction while preserving behavior.

1. Confirm your own Green and choose a useful Rename and Extract Method.
2. Run tests, read the diff and commit each passing operation separately.
3. Review names and boundaries, apply feedback and choose actual evidence.

Produce

rename and method-extraction commits, regression results and review findings

35 minutes • whole activity

Source activity: d03-s13

Review whether the name explains its role, the inputs and return fit the calculation, and the small diff and regression support behavior preservation. Keep remaining scope and missing operations explicit.

© 2026 Inspire | Confidential

INSPIRE AI

SHOWCASE

Show Refactoring Changes and Preserved Behavior



© 2026 Inspire | Confidential

Show What Became Clearer

Two learners show actual refactoring work. About five minutes each, including feedback; then compare.

Before	Where did the reader have to guess or filter details?
After	What does the new name or method boundary express?
Evidence	Show the separate diffs, inputs, return values and actual test results.

Focus on structure and preserved behavior. Include the saved version and help used.

© 2026 Inspire | Confidential

INSPIRE AI

BREAK

Take a Break



© 2026 Inspire | Confidential

YOUR TURN

When Is Another Refactor Worthwhile?

The reason for starting a change can also help us judge when to stop.

Use one real result from the showcase to decide.

We can extract another method or add another layer. What would it improve now?

© 2026 Inspire | Confidential

Clean Code and Simple Design

Fewest elements

Keep the parts that support these goals.
Extra layers can make change harder.

Clear intent

Help the next reader understand the purpose of the code.

No duplication

Keep one business rule from needing edits in several places.

Passing tests

Protect the current behavior with appropriate checks.

Source: Beck's Design Rules Discussed by Fowler

© 2026 Inspire | Confidential

Use Four Simple Design Questions

Tests pass	Are the checked current requirements supported?
Intent is clear	Do the names and boundaries explain the code?
Logic is shared	Does one rule avoid repeated, coordinated edits?
Elements are needed	Does each added part help the current design?

Beck's rules, as discussed by Fowler. Behavior and clarity take priority over reducing the number of elements.

© 2026 Inspire | Confidential

Compare a Useful Boundary with an Extra Layer

```
// A: Name the complete calculation
BigDecimal shipping = shippingFor(goods);

// B: Add one more method
BigDecimal shipping = shippingForOrder(goods);
private BigDecimal shippingForOrder(BigDecimal goods) {
  return shippingFor(goods);
}
```

Both use the same calculation. What meaning does the extra method add for this reader?

Illustrative continuation of shippingFor. Explain a current benefit before adding another layer.

© 2026 Inspire | Confidential

Save the Result When the Next Change Has No Clear Benefit

Current behavior has appropriate checks, intent is clear, and no useful duplication or unnecessary structure needs attention now.

> Can you explain a concrete benefit for another change?

If no clear benefit remains, save and stop this refactoring. Keep real behavior and evidence gaps explicit; revisit the design when a concrete difficulty appears.

© 2026 Inspire | Confidential

INSPIRE AI

PRACTICE

Practice Code Smell Recognition (Optional)



© 2026 Inspire | Confidential